UNIVERSIDAD AMERICANA

ESTRUCTURA DE DATOS

PILAS



MATERIAL DE CLASE N° 3.1

CONCEPTOS Y PRÁCTICAS BÁSICAS

Prof. Griselda Romero

Contenido

Estructuras (struct)

Pilas (stack)

- Teoría
- Usos
- Operaciones

Estructuras

Las estructuras son colecciones de variables relacionadas por un nombre. Las estructuras pueden contener variables de muchos tipos diferentes de datos a diferencia de los arreglos que contienen únicamente elementos de un mismo tipo de dato. Ejemplo:

```
struct Persona {
        string nombre;  //Miembro o campo de la estructura
        string direccion; //Miembro o campo de la estructura
        char sexo; //Miembro o campo de la estructura
        double salario; //Miembro o campo de la estructura
};
```

PILAS: Una pila es una estructura de datos lineal en la que los elementos se insertan y eliminan por el mismo extremo, llamado TOPE.

Principio: LIFO (Last In, First Out) → el último en entrar es el primero en salir.

Operaciones básicas

PUSH = Inserta un elemento en el TOPE.

POP = Elimina el elemento del TOPE.

TOP/PEEK = Consulta el elemento en el TOPE sin eliminarlo

Ejemplo 1: Insertar elementos en una pila, mostrar pila, eliminar un elemento, mostrar nuevamente la pila:

```cpp
#include <iostream>
using namespace std;

int main() {
    int pila[5];
    int tope = -1; // pila vacía
    int n, elemento;

    // Insertar elementos (PUSH)
    cout << "¿Cuántos elementos desea insertar en la pila? (máx 5): ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Ingrese elemento " << i+1 << ": ";
        cin >> elemento;
        pila[++tope] = elemento;
    }

```

int pila[5];

Se declara un arreglo de 5 enteros que funcionará como la pila.
Cada posición del arreglo representa un espacio donde se puede almacenar un elemento.
El tamaño fijo (5) significa que la pila puede contener como máximo 5 elementos → si se intenta insertar más, ocurre un overflow.

int tope = -1; // pila vacía
La variable tope indica la posición del último elemento insertado en la pila.
Inicialmente se asigna -1, lo que significa que la pila está vacía (no hay elementos).
Cada vez que se hace un PUSH, tope aumenta en 1.
Cada vez que se hace un POP, tope disminuye en 1.
Ejemplo:
Si tope = 0, hay un elemento en la pila (en la posición 0).
Si tope = 4, la pila está llena (5 elementos).
Si tope = -1, la pila está vacía.

int n, elemento;
Se declaran dos variables auxiliares:
n: se suele usar para almacenar la cantidad de elementos que el usuario quiere insertar en la pila.
elemento: se usa para guardar temporalmente el valor que el usuario ingresa antes de colocarlo en la pila

```
    // Mostrar pila actual
    cout << "\nPila actual: ";
    for (int i = tope; i >= 0; i--) {
        cout << pila[i] << " ";
    }
    cout << endl;

    // Eliminar un elemento (POP)
    if (tope >= 0) {
        cout << "\nElemento eliminado: " << pila[tope--] << endl;
    } else {
        cout << "\nLa pila está vacía, no se puede eliminar." << endl;
    }

    // Mostrar pila final
    cout << "\nPila final: ";
    for (int i = tope; i >= 0; i--) {
        cout << pila[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Ejemplo de la ejecución:

Output

```
¿Cuántos elementos desea insertar en la pila? (máx 5): 3
Ingrese elemento 1: 25
Ingrese elemento 2: 10
Ingrese elemento 3: 11

Pila actual: 11 10 25

Elemento eliminado: 11

Pila final: 10 25
```

Estructura del código:

1-El usuario indica cuántos elementos quiere insertar (máximo 5).

2-Se van almacenando en la pila con la operación PUSH.

3-Se muestra la pila completa desde el TOPE hacia la base.

4-Se elimina un elemento con POP.

5-Se muestra la pila final después de la eliminación.

Nota: overflow en estructuras de datos, especialmente en pilas, se refiere a un error de desbordamiento que ocurre cuando intentamos insertar un nuevo elemento en una pila que ya está llena.
Overflow: se produce cuando la pila ya alcanzó su capacidad máxima y se intenta hacer un PUSH.
Underflow : se produce cuando la pila está vacía y se intenta hacer un POP.
Estos errores son fundamentales para comprender las limitaciones de las estructuras de datos y la importancia de controlar los estados de la pila.

Ejemplo 2: Aquí sí definimos funciones auxiliares (push, pop, mostrar). Esto es parte de la modularización: separar el código en bloques reutilizables y claros

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;   // tamaño máximo de la pila
int pila[MAX];
int tope = -1;       // pila vacía

// Función PUSH: insertar elemento
void push(int elemento) {
    if (tope == MAX - 1) {
        cout << "Error: Overflow. La pila está llena." << endl;
    } else {
        pila[++tope] = elemento;
        cout << "Elemento " << elemento << " insertado en la pila." << endl;
    }
}

// Función MOSTRAR: recorrer la pila
void mostrar() {
    if (tope == -1) {
        cout << "La pila está vacía." << endl;
    } else {
        cout << "Contenido de la pila: ";
        for (int i = tope; i >= 0; i--) {
            cout << pila[i] << " ";
        }
        cout << endl;
    }
}

int main() {
    int n, elemento;

    cout << "¿Cuántos elementos desea insertar en la pila? (máx " << MAX << "):
        ";
```

```
        ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Ingrese elemento " << i+1 << ": ";
        cin >> elemento;
        push(elemento); // intento de inserción
    }

    // Mostrar pila final
    cout << "\nEstado final de la pila:" << endl;
    mostrar();

```

Ejemplo de ejecución:

```
Output                                                    Clear
¿Cuántos elementos desea insertar en la pila? (máx 5): 6
Ingrese elemento 1: 11
Elemento 11 insertado en la pila.
Ingrese elemento 2: 10
Elemento 10 insertado en la pila.
Ingrese elemento 3: 5
Elemento 5 insertado en la pila.
Ingrese elemento 4: 9
Elemento 9 insertado en la pila.
Ingrese elemento 5: 8
Elemento 8 insertado en la pila.
Ingrese elemento 6: 20
ERROR!
Error: Overflow. La pila está llena.

Estado final de la pila:
Contenido de la pila: 8 9 5 10 11
```

Ejemplo 3: integramos las acciones de una pila, con un menú de opciones:

main.cpp Share Run

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;
int pila[MAX];
int tope = -1; // pila vacía

// Función PUSH: insertar elemento
void push(int elemento) {
    if (tope == MAX - 1) {
        cout << "Error: Overflow. La pila está llena." << endl;
    } else {
        pila[++tope] = elemento;
        cout << "Elemento " << elemento << " insertado en la pila." << endl;
    }
}

// Función POP: eliminar elemento
void pop() {
    if (tope == -1) {
        cout << "Error: Underflow. La pila está vacía." << endl;
    } else {
        cout << "Elemento eliminado: " << pila[tope--] << endl;
    }
}

// Función TOP: devuelve el último elemento
int top() {
    if (tope == -1) {
        cout << "Error: La pila está vacía." << endl;
        return -1;
    } else {
        return pila[tope];
    }
}
```

```
38 void mostrar() {
39     if (tope == -1) {
40         cout << "La pila está vacía." << endl;
41     } else {
42         cout << "Contenido de la pila: ";
43         for (int i = tope; i >= 0; i--) {
44             cout << pila[i] << " ";
45         }
46         cout << endl;
47     }
48 }
49
50 int main() {
51     int opcion, elemento;
52
53     do {
54         cout << "\n--- MENU PILA ---" << endl;
55         cout << "1. Insertar (PUSH)" << endl;
56         cout << "2. Eliminar (POP)" << endl;
57         cout << "3. Consultar TOPE" << endl;
58         cout << "4. Mostrar pila" << endl;
59         cout << "5. Salir" << endl;
60         cout << "Seleccione una opción: ";
61         cin >> opcion;
62
63         switch(opcion) {
64             case 1:
65                 cout << "Ingrese el elemento a insertar: ";
66                 cin >> elemento;
67                 push(elemento);
68                 break;
69             case 2:
70                 pop();
71                 break;
72             case 3:
73                 if (top() != -1)
74                     cout << "Elemento en el TOPE: " << top() << endl;
75                 break;
76             case 4:
77                 mostrar();
78                 break;
```

```
79              case 5:
80                  cout << "Saliendo del programa..." << endl;
81                  break;
82              default:
83                  cout << "Opción inválida." << endl;
84          }
85      } while(opcion != 5);
86
87      return 0;
88  }
89
```

Tener en cuenta los ejemplos, toma nota de los errores que pudiste tener y la manera en la que los corregiste, eso siempre ayuda en las prácticas de programación.- Programación se aprende practicando.

